

Introduction to C++

Features of C++

Data type in C++

Variable in C++

Rule for naming variable

Operators / Types

Conditional statements

Loops in C++ / Types

Type of function in C++

Parameters passing in C++

Recursive function in C++ / Type

Arrays

+ the programs which mam told
us to make.

10/8/26

Introduction to C++

C++ is a general purpose programming language developed by Danish computer scientist Bjarne Stroustrup in 1979 at AT&T Bell Laboratories in New Jersey.

- C++ is an extension of C. It is widely used for developing high performing application, system software, games and embedded system support both processor and object oriented program.
- Provide fast execution speed in direct memory management capability/capabilities.
- Widely used in operating system, games, embedded system and competitive programming.

Features of C++

- 1 Object orientated programming
- 2 High performing executing
- 3 STL Libraries
- 4 Low level memory control
- 5 General purpose / High level language

C++ Program

• .cpp

include <iostream.h>

Header file

Using namespace std

name space ~~deklar~~ declaration

int main()

execution begin at main

{

cout << "Hello";

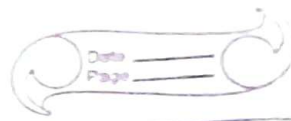
statement & comment (Hello)

return 0;

return value

}

Cin $\rightarrow$ Input
cout $\rightarrow$ output



Header file

include <iostream.h> adds input/output objects

(Cin, cout are the preprocessor (#)).

Preprocessor directive symbol. It runs before compilation

include: Keyword to import external files

<iostream.h> Input/Output stream library file.

note: modern C++ uses <iostream> without .h

Using name space Std; allow direct use of standard name
like "cout" without std::

using: Keyword to import a specific namespace

name space: A scope that prevents naming conflicts

std: standard C++ library

main() is the program entry point: execution starts here
and returns an integer with return zero means
successful execution.

() : Parentheses indicating this is a function

{ : opening brace code block starts.

Comments // single line, /*...*/ for multiple lines are
ignored by compiler and used only for code documentation.

statement contain executable code here cout << "Hello";
print the text on the screen using insertion
operation (<<)

Return The return zero: statement terminate the main()
function and indicate that the program execute successfully.

12/8/26

Code Practical

1. Addition

include <iostream.h>

include <conio.h>

int main()

{ clrscr();

int num1, num2, add;

cout << "enter the value num1 ";

cin >> num1;

cout << "enter the value num2 ";

cin >> num2;

add = num1 + num2;

cout << "addition is =" << add;

~~cout << "hello";~~

getch(); → get the character

return 0;

}

hold the
Screen

Output:

Enter the value num1 : 8

Enter the value num2 : 8

= 16

2. Subtraction

include <iostream.h>

include <conio.h>

int main()

{

clrscr();

int num1, num2, sub;

cout << "enter the value num1";

cin >> num1;

cout << "enter the value num2";

cin >> num2;

sub = num1 - num2

cout << "subtraction is =" << ^{Sub;}sub;~~cout << "hello";~~

getch();

return 0;

}

Output:

enter the value num1: 2

enter the value num2: 1

1

3 Division

```
#include <iostream.h>
#include <conio.h>
int main ()
{
    int num1, num2, div;
    cout << "enter the value num 1:";
    cin >> num1;
    cout << "enter the value num 2:";
    cin >> num2;
    div = num1/num2;
    cout << "division is = " << div;
    getch();
    return 0;
}
```

Output: Enter the value num 1: 3
Enter the value num 2: 3
: 1

4 Multipule

```
#include <iostream.h>
#include <conio.h>
int main ()
{
    int num1, num2, multip;
    cout << "enter the value num 1:";
    cin >> num1;
    cout << "enter the value num 2:";
    cin >> num2;
    multip = num1 x num2;
    cout << "multipule is = " << multip;
    getch;
    return 0;
}
```

Output: Enter the value num 1: 2
Enter the value num 2: 3
: 6

• How the run above code:

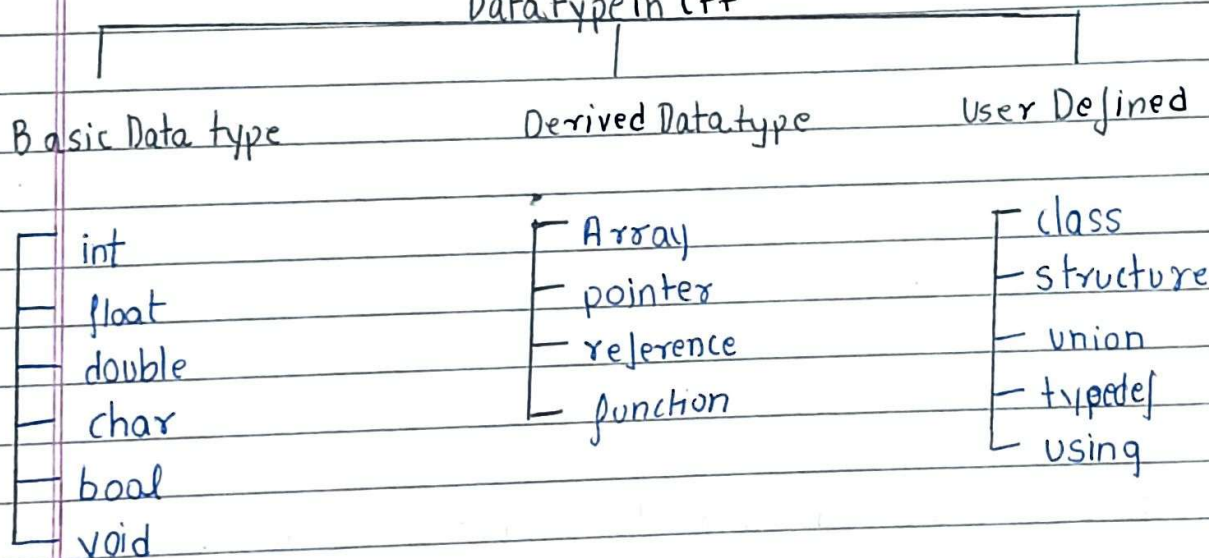
- 1 Write the code in a file such as hello world in cpp.
- 2 Use a c++ compiler (G++, Clant, MSVC) to compile source code
- 3 The compiler convert the source code into a executable file such as (Hello world.exe)
- 4 The operating system note executable file in memory.
- 5 The CPU execute the program instruction from main function.
- 6 The program display the output on the screen.

Data type in C++

Data type forms the foundation of data representation in C++ every variable must be associated with a data type. that ~~de~~ define the kind of value it can store and how that value handled by compiler.

1. Data type determine the memory ~~at~~ allocation, range of value and ~~oper~~ operation supported by variable.
2. Understanding diff. datatype is essential for writing correct efficient and maintainable C++ program.

Data type in C++





- 1 Integer data type: The int data type is used to store whole number.
- 2 Character data type: The char data type store a single char enclose in single quotes. Typically occupies 1 bit and store as ASCII value of character internally.
- 3 boolean data type: This is use to store logical value true (1) or False (0). The keyword use to define is bool. The size is 1 bit.
- 4 Floating datatype: (float): This is use to store decimal number it is declared using the float keyword. it occupies 4 bit, and store value approximately from 1.2×10^{-38} to 3.4×10^{38} .
- 5 Double data type: This store decimal number with high precision than float it occupies 8 bits and can store value approx: from 1.7×10^{-308} to 1.7×10^{308} .
- 6 void: The void represent the absence of value. we cannot create a variable of void type. it is use for pointer and functions that do not return any value using void keyword.

Derived Data type in C++

- 1 Array - An array stores multiple values of the same type in contiguous memory location.
`int arr[5] = { 1, 2, 1, 2, 5 };`
- 2 Pointer - They store the memory address of another variable
`int x=10;
int * ptr=&x;`

(String group of characters)



Reference: This act as an alias for existing variable

```
int x = 10;  
int &ref = x;
```

Function type: It also represent a derived data type ~~be~~ because have a specific return type and parameter type.

```
int add (int a, int b);  
{  
    return a + b;  
}
```

User define: Structure (struct)

a ~~structure~~ ~~struct~~ structure group related variable under 1 name.

```
struct student { string name;  
                int a;
```

Class: A class combine data and function in a single unit

```
class: space student { public: string name;  
                      int age;  
                      }
```

Union: A union allow multiple member to share the same ~~memoe~~ memory location.

```
union data { int i;  
             float f;  
             };
```

DATA TYPE SIZE and RANGE in C++

Data type	Size (Byte)	Range
int	4 Byte	-2147,483,648 to 2147,483,647
char	1 Byte	0 to 255
Float	4 Byte	$\pm 1.18 \times 10^{38}$ to $\pm 3.4 \times 10^{38}$ 7 decimal precision
Double	8 Byte	$\pm 2.23 \times 10^{-308}$ to $\pm 1.79 \times 10^{308}$ 15 decimal value
long Double	8, 12, 16 Byte	extend value more than Double
Bool	(1 byte) 0 (False) 1 (True)	
signed char		signed char = -128 to 127
unsigned char		- 0 to 255

Variable in C++

In C++ variable are the container used to store data in memory. variable define how data is store, access and manipulated. a variable has 3 components:

data type: Define the kind of data store (eg: int, string, float-).

Variable name: A unique identifier following C++ naming rules.

Value: The actual data assigned to variable.

Declaration: Creating a variable and giving a name is called variable definition.

the syntax of variable declaration

^{data type}
int x;
 ↪ variable

where type is the type of data that the variable can store name assign to variable.

Multiple variable of some data type can be defined:

Type name 1, name 2, ;

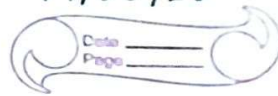
Initialising A variable contain a variable value to initiate value using assignment operator =.

int age = 20;
 ↪ reserved memory (ram)

→ Variable is to store data so that it can retrieve or update any time. Accessing can be done by simply using its assigned name. updating value using "=" assigned value.

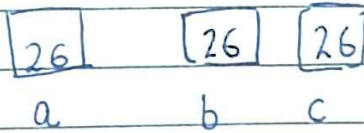
"=" assignment variable

14/08/26



Computer Lab

In memory



int stores 2 bytes of memory

printf :- To print the exact command

scanf :- To take input from user

use %d for integers & %f for float

clrscr → To clear screen

getch → This will stop the ~~op~~ output window once we run the code and keep it open till we press any character on keyboard.

```
# include <stdio.h>
```

```
# include <conio conio.h>
```

```
main ( )
```

```
{
```

```
int a, b, c;
```

```
clrscr();
```

```
printf ("enter any two numbers");
```

```
scanf ("%d %d", &a, &b);
```

```
c = a + b
```

```
printf ("Sum = %d", c);
```

```
getch();
```

```
}
```

F2 → save abc.cpp

Alt + F9 → compiler

Ctrl + F9 → Run

To change this code to C++

```
#include <iostream.h>
#include <conio.h>
void main()
{
clrscr();
int a, b, c;
cout << "enter any two numbers";
cin >> a >> b;
c = a + b;
cout << "Sum = " << c;
getch();
}
```

Output

```
enter any two numbers 22
                      12

Sum = 34
```

Note:

Ek hi file me Bhaut saare program likhna ke liye

Prg. hi starting aur ending me.

/*

/*

lga do fir new
program ushe
heerhe se likhe.

- Rules for naming variable
- The names given to variable are called identifier
a name only contain letters (A-Z or a-z), digits (0-9) and underscore (_)
- It should start with letter or underscore only.
- It is case sensitive.
- The name of variable should not contain any whitespace or special character. (/ , * , &)
- We cannot use C++ keywords (etc. int, float, char.) as variable name.

Variables are the names

Memory management of variable

When a variable is declared a fixed size memory block is declare to it and the variable that memory location

local variable may contain garbage value. if not initialised
Global and static variable they are initialised to zero.
Initialization assign a meaning full value using assign value
Variable represent specific memory location assigned by
various name.

Diff. variable may be stored in diff memory segment based on
their storage class.

Operators

C++ operators are the symbols that operate on value to perform
a specific mathematical or logical computation on given
value.

ex: plus (+) is the additional operator and does the
addition of ten and twenty and return thirty as results.



C++ Operator types

:: Scope resolution operator

::* Pointer-to-member declarator

->* Pointer-to-member operator

.* Pointer-to-member operator

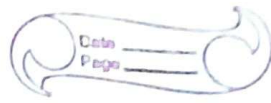
delete Memory release operator

endl line feed operator

new Memory allocation operator

setw Field width operator

Program for arithmetic operation



```
#include <iostream.h>
#include <conin.h>
void main()
{
    clrscr();
    int a, b, sm, sub, ml, div;
    cout << "enter the number first: ";
    cin >> a;
    cout << endl;
    cout << "enter the second number: ";
    cin >> b;
    cout << endl;
    sm = a + b;
    sub = a - b;
    ml = a * b;
    div = a / b;
    cout << "sum of numbers: " << sm;
    cout << endl;
    sm cout << "Difference of numbers: " << sub;
    cout << endl;
    cout << "multiplication of numbers: " << ml;
    cout << endl;
    cout << "division of numbers: " << div;
    cout << endl;
    getch();
}
```

Program for average

```
#include <iostream.h>
#include <conio.h>
void main()
{
clrscr();
float a, b, c, sum, avg;
cout << "Enter the first number:";
cin >> a;
cout << endl;
cout << "Enter the second number:";
cin >> b;
cout << endl;
cout << "Enter the third number:";
cin >> c;
cout << endl;
cout << "Enter the second
sum = a + b + c;
avg = sum / 3;
cout << "sum of numbers:" << sum;
cout << endl;
cout << "average of numbers:" << avg;
getch();
}
```

Relation Operators

Operators are use comparition ' $>$ ' check the right operator is greater.

equal to ($=$) check if both operants are equal

greater than ($>$) check if the first operant is greater than second greater than and equal to

($>=$) check the first operant is equal to second operator.

less than ($<$) the first operant is less than second operant.

less than equal to ($=<$) check the first operant is equal to the second operant.

Not equal to ($!=$) check the first operant not equal to the second operant and both are not equal.

Logical operators

they are use to combine two or more conditions or constrains to evaluate the original condition the result return a boolean value that is true or false.

	eq:	1 1 $\rightarrow$ 1 (T)	AND
Logical AND (&&)		1 0 $\rightarrow$ 0 (F)	
Logical OR ()		0 1 $\rightarrow$ 1 (T)	
Logical NOT (!)		0 0 $\rightarrow$ 0 (F)	
		exp1 exp2 result	

AND Return true value when all operant are true.

OR Return true if one operant is true.

1 1 $\rightarrow$ T (1)

1 0 $\rightarrow$ F (0)

0 1 $\rightarrow$ T (1)

0 0 $\rightarrow$ F (0)

NOT

Return true if the operand is false or 0 or zero

0 $\rightarrow$ 1 (True)

1 $\rightarrow$ 0 (False)

Bitwise Operator

They are used to work on bit level compiler first compare to byte level then calculation is performed all operand

Binary AND (&)

Copy a bit to result if it exists in both operands. return 1 if

both bit are 1

Binary OR (|)

Copies a bit to the result if it exists in any of the one operand return 1 atleast 1 byte is 1. (True value).

Binary XOR (^)

Copies the bit in the result if it is present either of the operand but both return 1 if bytes are diff.

1 1 $\rightarrow$ F (0)

1 0 $\rightarrow$ T (1)

0 1 $\rightarrow$ T (1)

0 0 $\rightarrow$ F (0)

One's complement (~)

0 $\rightarrow$ 1 (True)

1 $\rightarrow$ 0 (False)

change binary digit 1 to 0 and 0 to 1.

Left shift (<<)

shift the value to the left by the no. of bytes specified by the right operand.

0 1 1 0 1 1 left shift
0 1 1 0 1 1 spe shifting
0 0 1 1 0 1 1 Right shift

($\leftarrow$) equal to ~~the~~



Right shift ($\gg$)
Shift the value to the right by the number of byte.

Note

Only char and int data type can be use in byte wise operator

Assignment operator

They are use to assign value to the variable. we assign the value of right operand acc. to the ^{assignment} ~~right~~ operator used.

1) first assignment ($=$) :

Assignment the value on the right to the variable on the left.

Add and assignment ($+=$) : Add the right operand to the left operand and assign the result to the left operand.

Subtraction and assignment ($-=$) : Sub. the right operand to the left operand and assign the result to the left operand.

($*=$) Multiple and assignment: Multiply left operand to right and assign the result to the left operand.

($/=$) Divide and assignment: Divide left operand to right and assign the ~~use~~ result to the left operand.

(?) Ternary and conditional operator

conditional operator return a value based on the condition this operator takes 3 operands therefore they are ternary.

Syntax:

Expression 1 ? Expression 2 : Expression 3.

- The ternary operator determine the answer on the basis of the ~~so~~ evaluation of expression 1.
- If expression 1 is true then expression 2 get evaluated.
If expression 1 is false, then expression 3 get evaluated.

Miscellaneous operators

Size of operator - it is a unary operator compute the size of operand or variable in bytes.

Size of (char);

Size of (variable);

Common operator

It is a binary operator it is used as separation used as ~~sep~~ or use to evaluate its first operand and discard the result.
then evaluate second operand and return operand.

(*) Address Operator

use to find memory ~~operand~~ address where a particular operand is used.

(.) Dot operator

It is used to access the member of ~~struct~~ structure operator or class operand using these object.

Arrow operator

class structure or variable

It is use to ~~assi~~ access the [^] to the pointer.

Casting operator

It is use to convert the value of one data type to one data type. for ex: integer value of 'x'.

Basic Input and Output in C++

In C++ data is read and write using streams which are sequence of bytes:

- 1 Input stream - Data flow from a device like the keyboard to the computer memory.
- 2 Output stream - Data flow from memory to an output device like the screen.
- 3 These are define in the `<iostream>` header file
- 4 the most common stream object are "cin" for taking input and "cout" for taking output.

Standard Output stream

cout is the instance of the ostream class use to display the output on screen data is send to ~~see out~~ cout using insertion operator.

Standard Input stream

1 cin

It is the instance of the istream class use to read input of the keyboard.

2 << >>

The extension operator is use to cin to get data from stream to store data.

Control flow statements

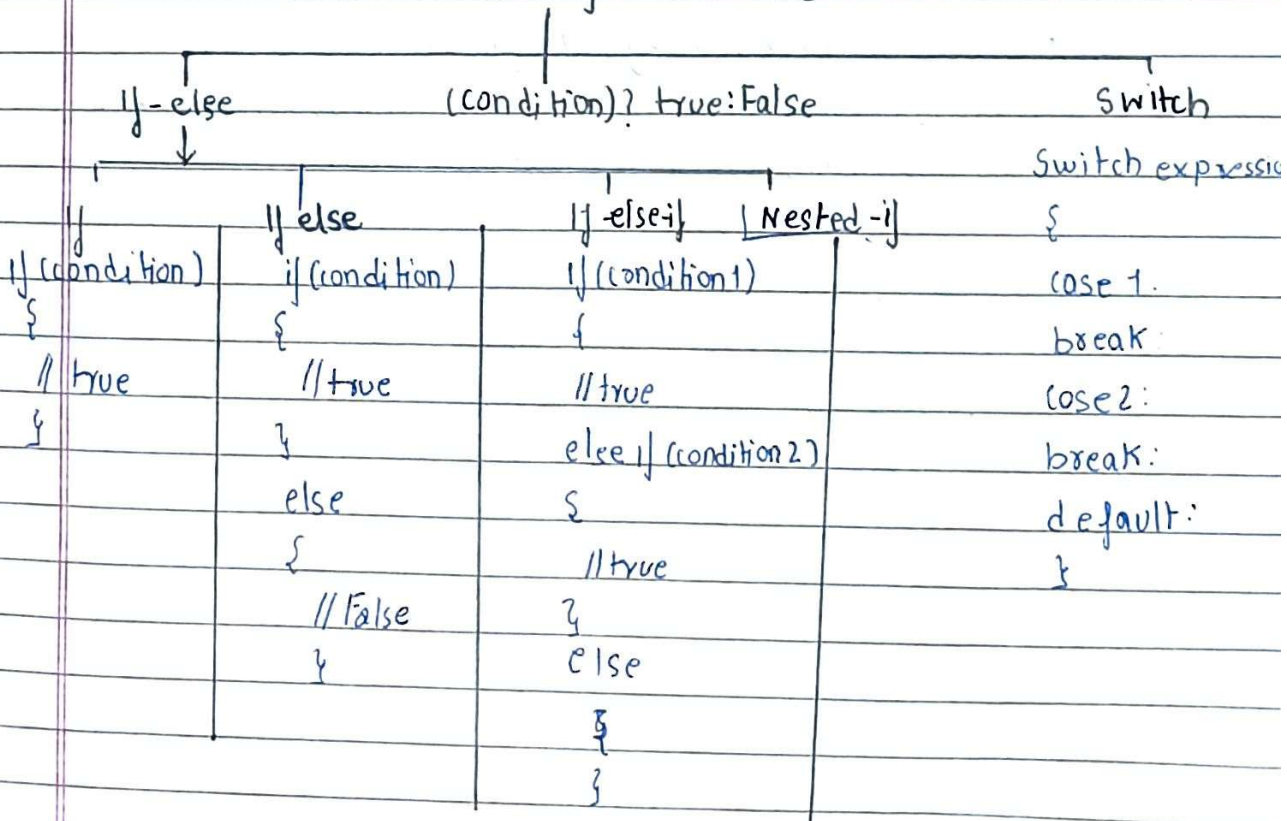
Control flow refers to the order in which statement in a program execute while program typically follow a sequential flow from top to bottom, there are scenario where we need more flexibility.

Control flow statements are fundamental statements of programming language that allow developer instructions are ~~exicub~~ executable and enable block of code multiple time and based on conditions, terminates the execution of ~~s~~ certain line of code.

Conditional statements

1. If statement in programming: If statement

Decision Making Statements in C++



If statement use to execute a block of code if a specified condition is true.

If else statment in programming - this statment is use to execute a specific condition is true and another block of statment if condition is false.

If else if in programming used to execute one block of code if a specified ~~condition~~ condition another of for othe statment and a default of the code if null is true.

Loops Switch

In C++ a switch statment is use to execute On block of code multipule option based on the value of expression

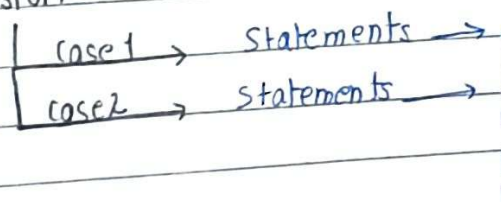
the variable or the evalution inside the switch (" "). {}

must be reserved integral type is expression

each case folows by a constant value and a colon : ;

Duplicate case values are not allowed.

Expression





Break

It is used for stopping execution. When break is used, the program will continue ~~execute~~ execute the code in case block automatically.

Default statement: If it serves as a fall back. It runs if none of the cases are matched.

if else statement

```
#include <iostream.h>
#include <conio.h>
int main()
{
    clrscr();
    int num;
    cout << "enter the number to be checked for odd or even" << endl;
    cin >> num;
    if (num % 2 == 0) { cout << "the number is even"; }
    else { cout << "the number is odd"; }
    getch();
    return 0;
}
```

output: enter the number to be checked for odd or even

4

the number is even

```
#include <iostream.h>
#include <conio.h>
void main()
```

```
{
    clrscr();
```

```
    int num;
```

```
    cout << "enter number";
```

```
    cin >> num;
```

```
    if (num % 2 == 0)
```

```
    {
```

```
        cout << "number is even";
```

```
    }
```

```

else
{
cout << "number is odd";
}
getch();
}

```

```

int a, b, c;
Enter printf ("Enter the no. a and b: ");
scanf ("%d %d", &a, &b);
if (a + b == 10)
    printf ("Sum is 10");
else
    printf ("Sum is not 10");

```

a = b; ~~b = c~~

b = c;

c = a;

root value after 5

Swaping of Numbers

- Header file

```
main()
```

```
{
```

```
int a,b,c;
```

```
cout << "
```

```
cin >>
```

```
a = b;
```

```
b = c;
```

```
c = a;
```

```
cout <<
```

- # include <iostream.h>

```
# include <conio.h>
```

```
int main ()
```

```
{
```

```
clrscr();
```

```
int a,b,c;
```

```
cout << "enter a: " << endl;
```

```
cin >> a;
```

```
cout << "enter b: " << endl;
```

```
cin >> b;
```

```
cout << "value after swopping" << endl;
```

```
a = b;
```

```
b = c;
```

```
c = a;
```

```
cout << "a=" << a << endl;
```

```
cout << "b=" << b << endl;
```

```
cout << "c=" << c << endl;
```

```
getch();
```

```
return 0;
```

```
}
```

- #include <iostream.h>

#include <conio.h>

void main ()

{

clrscr ();

int i = 1, n, sum = 0;

if (i <= n)

{

sum = sum + i;

i = i + 1;

}

cout << "sum = " << sum;

getch ();

}

Loops in C++

In C++ there are ~~se~~ situations where a task means to repeat multiple times.

For example to print "hello world" 5 times we would write the cout statement 5 times in loops. We have used one of the loops to print hello world 5 times.

You can see there code is reduced.

Loop structure is used for the iteration or repetition of code.

~~For loop~~

Types of loops

1. For loop : The for loop is the entry control loop which means that it checks whether the test condition is true before executing the statement inside it.

Syntax:

```
for (initialization ; condition Test ; updating)
{ // stmt;
}
```

print

```
→ int i;
   for (i = 0; i <= 5; i++)
   {
       cout << "i";
   }
```

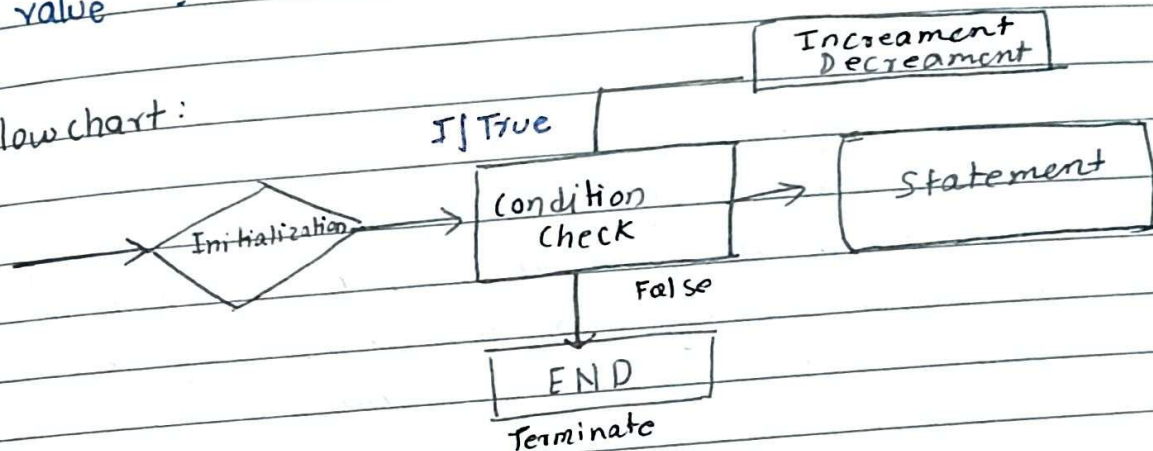
To print the number

1. Initialisation : Initialise the variable to some initial value
2. Test condition: This specifies the test condition if the condition evaluates to true then the body of the loop is executed if it evaluates to false then

The loop is terminated

Update expression: After the execution loop's body ^{this} if expression increment or decrement the loop variable by some value.

Flowchart:



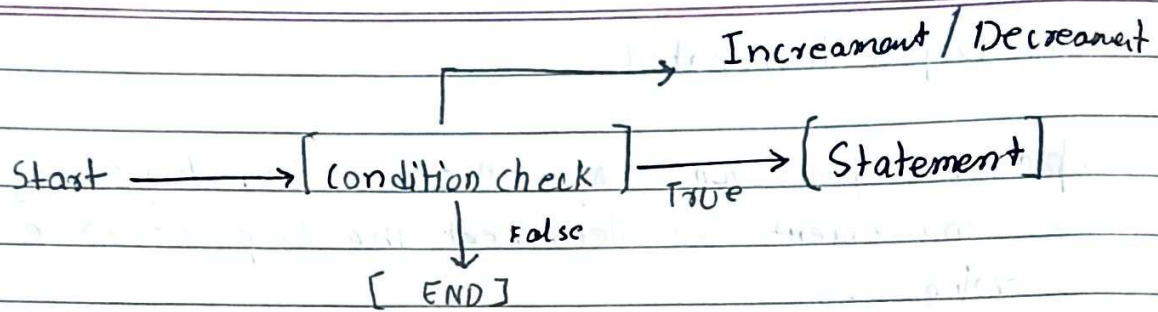
while ~~while~~ loop: This loop is called entry control loop which is used in situation where we don't know the exact number ^{iteration} of the loop.

Syntax:

```
while (condition)
{ // Body of the loop ;
  // update expression ;
}
```

eg.

```
while (i <= 5)
{
  cout << "i";
  i++;
}
```



• Do while loop:

The do while loop is the exact control loop which means the condition is checked after the executing body. In a do while loop the statement is executed in one time.

```

do
{
    // Body of the loop ;
}
while (condition);
a++;
  
```

• Diff. of all loops

	for loop	while loop	do-while loop
Control type	Entry controlled	Entry controlled	Exit controlled
Condition check	checked at the beginning	checked at beg. the end	checked at the end
Min Execution	0 (if condition is false)	0 (if condition is false)	1 (Always runs once)
Best Used for	Fixed/known iterations	Unknown no. of iterations	Menus or user inputs

Syntax
Style

~~condition~~ Compact
~~at header~~ (All-in-one-line)

condition
at header

do block ends
with while (),

Prime / non prime

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int num;
```

```
int i=2;
```

```
bool isPrime = true;
```

```
cout << "Enter a positive integer: ";
```

```
cin >> num;
```

```
if (num <= 1) {
```

```
isPrime = false;
```

```
}
```

```
while (isPrime && (i*i <= num))
```

```
{
```

```
if (num % i == 0)
```

```
{
```

```
isPrime = false;
```

```
}
```

```
i++;
```

```
}
```

```
if (isPrime) {
```

```
cout << num << " is a prim no. " << endl;
```

```
else {
```

```
cout << num << " is a non prime (composite) number. "
```

```
}
```

```
return 0;
```

Functions ↗ Pre-defined
→ user defined

```
#include <iostream.h>
#include <conio.h>
int main()
{
    getch();
    return 0;
}
```

```
#include <iostream.h>
#include <conio.h>
int addition (inta, intb)
{
    int sum;
    sum = a + b;
    return;
}
int main()
```

Lab (Factorial)

```
#include <iostream.h>
#include <conio.h>
void main()
{
    clrscr();
    int n, fac;
    cout << "Enter the no. to find factorial = ";
    cin >> n;
    fac = 1;
    while (n >= 1)
    {
        fac = fac * n;
        n = n - 1;
    }
    cout << "Factorial = " << fac;
    getch();
}
```

A function is used following by () and pretests, passing required argument if any, which execute code inside the function.

note: function call allow you to be we use code easily. you can call the same function multiple times with diff inputs to perform repeated task in a sturture way.

Parameters and argument

A function ~~can~~ can except input values calle arguments which are passed during function call. and recived through typed parameters.

Define inside the function brackets.

Syntax:

```
return type function name (Parameter 1 Parameter 2)
{
    //
    function Body;
    // return;
}
```

note: A function can only take many argument as specific function defination and it is compulsory to pass them will calling it also they should be of same type as in the function defination.

Type of function in C++

~~Basic~~ Based on origin

1. Library functions - These are built in function provided by C++ standard libraries such as `sqrt()`, `main()`, `clrscr()`, `getch()`; you can use them by including header files like `<iostream.h>`, `<conio.h>` using namespace `std`;

User define function

These are function created by the programmer to create by program.

2. Based on input and return type: User define function can be further classified ~~as~~ based on whether they accept or return value.
 - 1. no parameter, no return value. - The function perform a task but does not take input or return any thing.
 - 2. Parameters, not returns value the function take input but does not return result.
 - 3. No parameter return value. Function return a result but does not take any input.
 - 4. The function take input and return result

Note:

These type help you define functions acc. to the task we need to perform choosing the right form included improves code feasibility and clarity.

→ Parameters Passing in C++ :

In C++ data is passed # to function during a call to perform a operation. This data is known as Parameters or arguments.

1 Formal Parameters :

Variable used in Parameter list of a function as placeholder also called only parameters.

2 Actual Parameters :

The expression or values passed in during a function call also called arguments.

There are three diff. methods using which we can pass parameters to a function in C++.

Pass by Value :-

In Pass by value method a variable's value is copied and then passed to the function.

As a result, any changes to the Parameter inside the function will not affect the variable's original value in the caller.

↳ main function

Pass by reference : (&)^{→ Sign of refrence → address}

In Pass by reference method, Instead of passing the value of the argument we pass the reference of an argument to the function. This allows the function instead of refrence.

Pointer is used to store address of another variable

Pointer is used to store the address of any other variable

```
int add() :
int &a;
int * ptr = &a;
int * cout << "&a"
cout << " * ptr";
```

These will give same ans

* [Dereferencing] me ptr ka address store hota ha means address ha bhi address call kara rhee!

Syntax: ** ptr

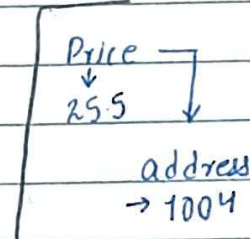
a ptr
[10] [1001]

float Price (25.5)

```
{
    int * ptr = &price;
    cout << "&price";    -> 1004
    cout << " * ptr";    -> 1004
    cout << " float";    -> 25.5
}
```

If will print:

Running the code:-



↳ In memory

Memory address are in hexadecimal form i.e. address of any variable are in this form.

& → Bit wise operator

&& → AND

Pointer is used to store the

address of any other variable.

Recursive function in C++

Recursion is a programming technique where a function call itself repetitively to solve a problem. The recursive call continues until a specific base condition is met which stops the recursion.

function call itself is known as recursive function.
A self call of the function is called a recursive call.

When a recursive function is called it executes a set of instructions and then calls itself to execute the same set of instructions with a smaller input.

Recursive Case

It is the way which the recursive call is present in the function.

Base condition

The base condition terminates the recursion.

This structure allows a problem to be broken down into simpler versions of that problem, making recursion a powerful tool for solving problems.

```
recursive function {  
    {  
        // recursive call : // calling self function  
    }  
}
```

Working of recursive function

To understand how recursion works internally, it is important to understand the behavior of the call stack during recursive calls.

Whenever a function calls itself, the current function state is stored.



store in stack and the new recursive call continue until the base case is reached.

- The descending phase represents recursive call going deeper into the function.
- The ascending phase begin after reaching the base case where function return one by one from the stack.

Memory

Type of Recursion in C++

Recursion is a process where a function call itself either directly or indirectly for small data. In C++ recursion ~~can~~ occurs by ~~with~~ writing a function that include a call within its body.

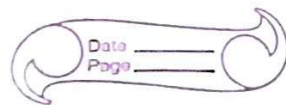
Recursion in C++ can be classified in to the following type.



Direct Recursion :-

Direct Recursion occurs when a function call itself directly from within the body. This is the most common and straight forward form of recursion. Direct Recursion can be divided into :-

cout → console output
cin → console input
getch → get the character
↳ used to hold output screen



1 Head Recursion:

In Head recursion the recursive call happens before any processing in the function. The function calls itself first and processes later.

2 Tail Recursion:

In tail recursion the function processes first and the recursive call is the last operation. Tail recursion can be more memory efficient and may benefit from compiler optimisation.

3 Tree recursion:

Tree recursion happens when a function calls itself more than once within its body forming a tree-like structure.

→ Indirect Recursion:

In Indirect recursion a function does not call itself directly; it calls another function that eventually calls the first one, creating a chain of calls. For example, function A calls function B which calls function A again, forming indirect recursion.

lab (Palindrome)



→

```
#include <iostream.h>
#include <conio.h>
int main()
{
    clrscr();
    int i, rev=0, original;
    cout << "enter the integer";
    cin >> i;
    original = i;
    while (i > 0)
    {
        rev = (rev * 10) + (i % 10);
        i = i / 10;
    }
    if (original == rev)
        cout << "no. is palindrome";
    else
        cout << "no. is not palindrome";
    getch();
    return 0;
}
```

→

Switch (Days)

```
int day;
cout << "enter the number (1-7)";
cin >> day;
switch (day)
{
    case 1: cout << "monday";
            break;
    case 2: cout << "tuesday";
            break;
    case 3: cout << "wednesday";
            break;
    case 4: cout << "thursday";
            break;
    case 5: cout << "Friday";
            break;
    case 6: cout << "saturday";
            break;
    case 7: cout << "sunday";
            break;
    default: cout << "Invalid";
}
}
```

```
#include <iostream.h>
#include <conio.h>

int main()
{
    void main(int x)
    {
        int a = 0, b = 0, sum = 0;
        cout << "Enter a no. ";
        cin >> a >> b;
        sum = a + b;
        return 0;
        getch();
    }
}
```

```
#include <iostream.h>
#include <conio.h>

f(int x, int y)
void main()
{
    clrscr();
    int p = 10, int q = 20;
    f(p, q);
    getch();
}

f(int w, int z)
{
    int a, b, sum = 0;
    a = 10;
    b = 20;
    S = a + b + w + z;
    cout << S;
    return 0;
}
```

#

#

```
f()
void main()
{
    clrscr()
    f();
    getch()
}
```

```
f()
{
    int a, b, sum;
    a = 10;
    b = 10;
    sum = a + b;
    cout << S;
    return 0;
}
```

The Array

Array in C++ is a fixed size collection of elements of the same datatype stored in contiguous memory location. Each element can access using its index.

array indexing start from 0. so the first element is at index zero. the last element is at (size-1)

The size of a built-in array is fixed after declaration can be ~~delimited~~ determined using size of the operation when the array is in the same scope. Scope.

Declaration of An array:

We can create or declare an array by simply specifying the data type first then the name the array with its size inside `[]` square bracket.

Syntax

`datatype arrayname [];`

1. Specifying the type of element ~~store~~ stored in the array
2. Array name is the name of the array.
3. size specify the no. of elements that can store by the array

`int arr[5]` `arr[5]` (size of array = 5)



memory allocated

Initialisation of array

It means assignment initial values to array element we can initialise array by the providing the value inside curly braces `{}` which are assign array element in order

`int arr[5] = {10, 11, 12, 13, 14};`

Date _____
Page _____

These values are assigned sequentially this means that the first element index will be (0), (1) second will be 1, (11) so on.

The element in array can not be more than the size of the array but can less and this is partial initialization.

```
int arr[5] = { 2, 4, 6 }  
[2][4][6][ ][ ]
```

Operation on array element

These are the common actions performed on array to store, update, access, modify, manage data efficiently.

Access array element

Element of an array can be accessed by their position (called index) in the sequence. In C++ the indexing of an array starts from zero instead of 1. We just have to pass this index inside the ([]) with the array.

It is imp. to note that index can not be negative or greater than size of the array - 1.

Update array element

To change the address a particular index is an operator ~~ex~~ with new values as right hand expression by accessing the array element
`arr name [index] = value.`